

Estructura de Datos y Algoritmos I

Profesora: Elba Karen Sáenz García

Greedy Algorithms

Algoritmos Ávidos



Algoritmos ávidos

- También llamados voraces o golosos (Greedy Algorithms)

Algoritmos ávidos

- Dado un problema con n *entradas* el método consiste en obtener un subconjunto de éstas que satisfaga una determinada restricción definida para el problema.
- Cada uno de los subconjuntos que cumplan las restricciones diremos que son soluciones ***prometedoras***.
- *Una solución **prometedora** que maximice o minimice una función objetivo* la denominaremos solución óptima.

Algoritmos ávidos

- Un problema es susceptible a resolverse por un algoritmo ávido si tienen los siguientes elementos presentes en el problema:
 - 1-Un conjunto C de *candidatos*, que corresponden a las n entradas del problema.
 - 2- Una función ***de selección voraz*** que en cada momento determine el ***candidato idóneo*** para formar la solución de entre los que aún no han sido seleccionados ni rechazados.

Algoritmos ávidos

- 3-**Función factibilidad:** Una función que compruebe si un cierto subconjunto de candidatos es ***prometedor*** . *Entendemos por prometedor que sea posible seguir añadiendo* candidatos y encontrar una solución.
- 4-Una ***función objetivo*** *que determine el valor de la solución hallada. Es la* función que queremos maximizar o minimizar.
- 5-Una función solución que compruebe si un subconjunto de estas entradas es solución al problema, sea óptima o no.

Funcionamiento Algoritmos ávidos

- 1- Tratará de encontrar un subconjunto de candidatos tales que, cumpliendo las restricciones del problema, constituya la solución óptima.
2. Se trabajará por etapas, tomando en cada una de ellas la decisión que le parece la mejor, sin considerar las consecuencias futuras, y por tanto escogerá de entre todos los candidatos el que produce un óptimo local para esa etapa, suponiendo que será a su vez óptimo global para el problema.

Algoritmos ávidos

3-Antes de añadir un candidato a la solución que está construyendo comprobará si es prometedora al añadirlo. En caso afirmativo lo incluirá en ella y en caso contrario descartará este candidato para siempre y no volverá a considerarlo.

4- Cada vez que se incluye un candidato comprobará si el conjunto obtenido es solución.

Algoritmo Genérico



C =conjunto de candidatos

S =Conjunto solución, inicialmente esta vacío

x = es un elemento seleccionado del conjunto C

Voraz (C)

Inicio

$S = \emptyset$

Mientras (C sea diferente de 0) y $\text{solucion}(S) == \text{Falso}$ hacer

Inicio

$x = \text{seleccion}(C)$

$C = C - x$

Si $\text{prometedor}(S+x)$ entonces

$S = S + x$

Fin Si

Fin Mientras

Si $\text{solución}(S) == \text{Verdadero}$ entonces

Retorna S

En otro caso

Retorna “No existe solución”

Fin

Ejemplo Mínimo número de monedas

Se pide crear un algoritmo que permita a una máquina expendedora devolver el cambio mediante el menor número de monedas posible.

Considerando que el **número de monedas es limitado, es decir, se tiene un número concreto de monedas de cada tipo.**



Análisis

Candidato: conjunto finito de monedas de, por ejemplo, 50 , 25 , 5 y 1 unidades, con un número de monedas ilimitado o limitado.

Solución: conjunto de monedas cuya suma es la cantidad a pagar.

Prometedor: la suma de las monedas escogidas en un momento dado no supera la cantidad a pagar.

Función de selección: la moneda de mayor valor en el conjunto de candidatos aún no considerados.

Función objetivo: número de monedas de cada denominación utilizadas en la solución.

Ejemplo

Se tienen monedas de denominación 50 , 25 , 5 y 1 y hay que devolver la cantidad de 110.

- Se toma primero una moneda de 50, quedando 60 de resto.
- Como 50 todavía es menor que 60, se toma otra moneda de 50, y el resto es 10,
- Se devuelve una moneda de 5 ya que 50 y 25 son mayores a 10 y se desechan
- Se toma otra de 5 terminando el problema de forma correcta.

Solución : Cambio dos de 50 y dos de 5.

Solución

denMonedas=[50 , 25 , 5 , 1]

numMon=[inf, inf, inf, inf]

Sol=[2,0,2,0]

Cambio = $2*50+2*5$

Número de monedas limitado

- Si hay un número de monedas de cada denominación limitado, entonces considerar que sea posible tomar monedas de cada denominación para formar la solución

Ejemplo

- Si se tienen 3 de 50, 4 de 25, 1 de 5 y 6 de 1.
- Y se tiene que devolver 110

Siguiendo el método del algoritmo voraz :

- 1- Se toma primero una de 50 quedando 60
- 2- Se toma otra de 50 y quedan 10
- 3- devolver una moneda de 5 , ya que 50 y 25 son > 10
- 4- Se toma otra de 5 , pero ya no hay devuelve 5 de 1

Algoritmo

Entradas

denMonedas

numMon,

Importe

N

Salida:

SolCambio:

Algoritmo

cambio(denMonedas, numMon, importe,n)

Selección(denMon,NumMon)

i=0

Mientras importe>0 y i<n

Si denMonedas[i] <= importe y numMon[i]>0

numMon[i]= numMon[i]-1

solCambio[i]=solCambio[i]+1

importe=importe-denMon[i]

Si no

i=i+1

Fin Si no

Fin Mientras

Si importe >0:

No se puede dar el cambio correcto

Fin Si

retornar solCambio

Características algoritmos Ávidos

- Toman decisiones basadas en información inmediatamente disponible, sin importar consecuencias futuras.
- Se caracterizan por la rapidez en que encuentran una solución (cuando la encuentran), la cual no siempre es la mejor.
- Normalmente son utilizados para resolver problemas en los que la velocidad de respuesta debe ser muy alta.
- Se aplican principalmente a problemas de optimización. Donde se maximiza o minimiza una función objetivo.

Problema de la mochila:

- El problema consiste en que se tienen:
n objetos, cada uno con un peso (w_i) y un valor (v_i)
- Una mochila en la que se meterán objetos o fracciones x_i de ellos
, y la cual tiene una capacidad de peso máximo W .

$$0 \leq x_i \leq 1$$

Problema de la mochila:

- El objetivo es llenar la mochila, maximizando el valor de los objetos transportados, y respetando la limitación de capacidad máxima W .

$$\sum_{i=1}^n x_i v_i$$

$$\sum_{i=1}^n x_i w_i \leq W$$

Diseño de la solución:

- **Candidatos:** Cada uno de los n objetos de partida.
- **Función solución:** Se tiene una solución si hemos introducido en la mochila el peso máximo W , o si se han acabado los objetos.
- **Función selección:** Escoger el objeto más prometedor.
- **Función factible:** Será siempre cierta (podemos añadir trozos de objetos).
- **Añadir a la solución:** Añadir el objeto entero si cabe, o en otro caso la proporción del mismo que quede para completarla.
- **Función objetivo:** Suma de los valores de cada candidato por la proporción seleccionada del mismo.

Función selección

¿Qué criterio se puede usar para seleccionar el objeto más prometededor

- Posibles criterios para seleccionar el mejor objeto de los restantes:
 1. El objeto con más valor v_i : $\max (v_i), i= 1, \dots, n$
 2. El objeto menos pesado w_i (para poder añadir muchos objetos): $\min(w_i), i= 1, \dots, n$
 3. El objeto con mejor proporción v_i/w_i (valor por unidad de peso): $\max (v_i/w_i) i= 1, \dots, n$

Ejemplo:

$$W=17 \quad n=3$$

$$(v_1, v_2, v_3) = (40, 36, 22)$$

$$(w_1, w_2, w_3) = (15, 12, 8)$$

Selección 1

- El objeto con más valor v_i $W=17$

$$(v_1, v_2, v_3) = (40, 36, 22)$$

$$(w_1, w_2, w_3) = (15, 12, 8)$$

Escoger objeto 1 , luego 2

$$W = 15(1) + 12(2/12) + 0 = 17$$

$$V = 40(1) + 36(1/6) + 0 = 41$$

$$X = (1, 1/6, 0)$$

Selección 2

- El objeto menos pesado w_i $W=17$

$$(v_1, v_2, v_3) = (40, 36, 22)$$

$$(w_1, w_2, w_3) = (15, 12, 8)$$

Escoger objeto 3, luego el 2 y al final el 1

$$W = 8(1) + 12(9/12) + 8(0) = 17$$

$$V = 22(1) + 36(3/4) + 0 = 49$$

$$X = (0, 3/4, 1)$$

Selección 3

- El objeto con mejor proporción v_i/w_i

$$(v_1, v_2, v_3) = (40, 36, 22)$$

$$(w_1, w_2, w_3) = (15, 12, 8)$$

$$v_i/w_i = (40/15, 36/12, 22/8)$$

Escoger primero objeto 2, luego objeto 3,

$$W = 12(1) + 8(5/8) + 15(0) = 17$$

$$V = 36(1) + 22(5/8) + 40(0) = 49.75$$

$$X = (0, 1, 5/8)$$

mochila(peso[],valor[],W,n)

INICIO

Algoritmo

crear vector o lista Sol, inicializarla en 0

resto=W

k=0

Funselección(peso,valor)

Mientras $k < n$ **y** $\text{peso}[k] \leq \text{resto}$

$\text{sol}[k]=1$ /*selección del k ésimo elemento ya
ordenado*/

$\text{resto}=\text{resto}-\text{peso}[k]$

$k=k+1$

Fin mientras

Si $k < n$ hacer

$\text{sol}[k]=\text{resto}/\text{peso}[k]$

Fin Si

retornar sol

FIN

Entradas:

Pesos de los objetos: peso[]

Valor de los objetos: valor[]

Objetos :O[]

Peso máximo Mochila: W

Número de objetos: n

Salidas:

Vector o lista que contiene 1
o fracciones que indican si se
coloca el objeto completo a
fracción del mismo : sol[]

Problema Selección de Actividades

- Se tiene un conjunto de actividades que requieren el uso de un recurso(asistencia a la actividad) , de manera exclusiva, durante un intervalo de tiempo. Es decir se tiene que elegir entre el conjunto de actividades
 - Para cada actividad, se conoce su hora de comienzo y su hora de finalización.
 - Se puede asistir a todas las actividades que queramos.
 - Sin embargo, hay actividades que se solapan en el tiempo y no se puede estar en dos sitios a la vez.

Problema Selección de Actividades

- El objetivo es asistir al mayor número de actividades posible y minimizar el tiempo que estamos ociosos.

Se tendrá como entrada

Un conjunto C de n actividades con:

s_i = tiempo de comienzo de la actividad i

f_i = tiempo de finalización de la actividad i

Salida:

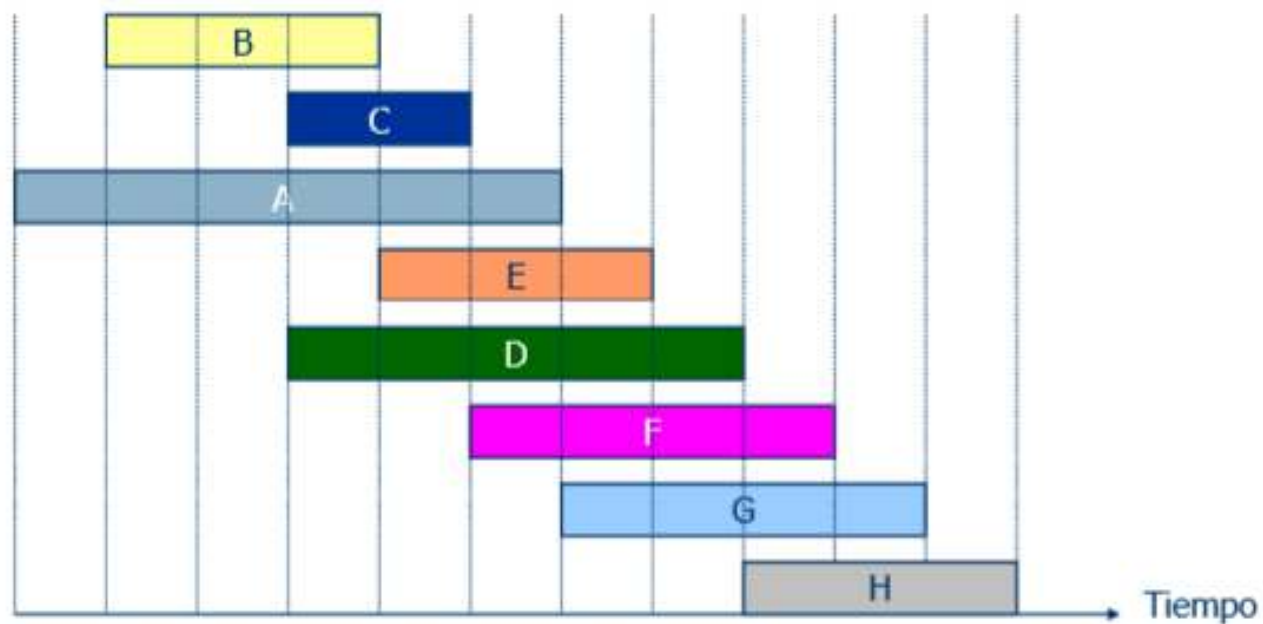
- subconjunto S de actividades compatibles de tamaño máximo (esto es, actividades que no se solapen en el tiempo).

Crterios de seleccin

Orden en el que se pueden considerar las actividades:

- Orden creciente de hora de comienzo.
- Orden creciente de hora de finalizacin.
- Orden creciente de duracin: fi -si

Selección por tiempo de finalización



SelecciónActividaes(C, si, fi)

Inicio

Ordenar C en base al criterio de selección

S[0]= C[0] //Seleccionar primera actividad

i=1

prev=0

Mientras i< numero de tareas en C

 x=C[i]

 Si s[i] >= f[prev]

 prev++

 S[prev]=x

 Fin Si

 i++

fin Mientras

Fin